1. Syntax Errors

These are mistakes in the "grammar" of the programming language. They happen when your code breaks the rules of how commands must be written.

When they happen: Before the program runs (during compilation or interpretation). The program will not start at all until you fix them.

Causes: Typos, missing punctuation, or unclosed brackets and quotes.

Examples:

1. Missing a semicolon; at the end of a line in C++ or Java.
2. Unclosed parenthesis: print ("Hello")

How to fix: Read the error messages (the compiler usually points to the exact line) and use a modern code editor (IDE) that highlights mistakes as you type.

2. Runtime Errors

These errors happen when your code grammar is perfect, but the program tries to perform an impossible or illegal operation while running.

When they happen: While the program is running. The program starts fine but suddenly crashes or throws an exception.

Causes: Bad user input, missing files, or trying to access memory that does not exist.

Examples:

1. Division by zero: x = 5 / 0.
2. Index out of range: Trying to get the 10th item from a list that only has 3 items.
3. Missing file: The program tries to open an image that was deleted.

How to fix: Use exception handling blocks (like try-catch in Java/C++ or try-except in Python) to prevent crashes. Always validate user inputs before using them.

3. Logic Errors

These are the trickiest bugs. The code grammar is perfect, and the program runs without crashing, but it gives the wrong result because your plan or formula is wrong.

When they happen: After the program runs, when you check the output. The computer does exactly what you told it to do, not what you wanted it to do.

Causes: Wrong math formulas, incorrect conditions, or misunderstanding the task.

Examples:

1. Writing subtraction instead of addition: total = a - b (instead of +).
2. Using the wrong loop condition, causing an infinite loop.
3. Wrong math order: avg = a + b / 2 (instead of (a + b) / 2).

How to fix: Use a debugger to step through the code line by line, print variable values to the console (logging), and write automated unit tests.